

AgentShift: Execution Mobility for Suspended LLM Agents

Anonymous Submission

Abstract

LLM agents alternate between model inference and external tools. After an LLM turn completes, an agent may wait for a tool while its accumulated prefix remains cached on one serving engine. This *suspended-but-warm* state creates a placement conflict: sticky routing preserves the prefix but concentrates returning agents, whereas rerouting relieves load but reconstructs history. Existing routing, retention, offload, and active-request migration do not transfer a suspended agent’s warm state together with its right to execute.

AgentShift makes suspended executions mobile across identical LLM engines. It separates durable continuation from reconstructable KV state, atomically installs the completed prefix on a destination, and commits an epoch-fenced ownership handoff during the blocked interval. A source shadow provides fast recovery until the destination produces its first token.

We implement AgentShift in SGLang and evaluate Qwen3 models with tensor parallelism up to four. At a 32K-token prefix and a 500 ms gap, AgentShift matches the sticky latency floor while reducing post-tool latency by $24.04\times$ over rerouting and $2.41\times$ over the same transfer started on return. In an eight-agent 32K burst, it relocates half the owners with full prefix hits and is $7.63\times$ faster than rerouting. Warm-pool scale-out and scale-in are $3.71\times$ and $6.43\times$ faster than semantic rerouting. Within our fault model, epoch fencing prevents double advancement and source-shadow recovery is $10.9\times$ faster than reconstruction.

1 Introduction

Long-lived LLM agents execute a sequence of model turns separated by external events. A coding agent may generate a command, wait for a compiler, inspect the result, and then issue another LLM request whose prompt contains the full interaction history. Serving systems exploit this continuity by retaining the completed prefix KV cache and routing the next turn back to the same engine. This policy avoids histor-

ical prefill, but it also binds future computation to the engine that holds the cache.

Tool execution exposes a state that request-oriented serving does not model: the agent is *suspended but warm*. Its previous LLM request has finished, no decode request is active, and the agent cannot make progress until the tool completes. Nevertheless, its completed prefix, logical continuation, pending tool result, and future execution placement remain live. Real traces show that this state matters: 29.3% of 245,555 Kimi K2.5 requests contain at least 16K input tokens and 12.5% contain at least 32K. In FlowPrefill sessions, the 90th-percentile prefix grows from 5.5K tokens at the first turn to 11.8K at the tenth. Long-lived agents therefore accumulate increasingly expensive state while repeatedly entering intervals in which they cannot execute.

Suspended warm state creates a locality–placement conflict. Sticky routing preserves locality, but correlated tool completions can return many agents to one engine while another engine is idle. Least-loaded rerouting uses the idle engine, but it reconstructs every historical token there. Program-aware routers such as Agentix improve placement decisions but still choose between these two outcomes [6]. TTL retention and source-side offload preserve or reclaim memory on the original engine [2, 5]; shared KV tiers can make state available elsewhere [1]. None of these mechanisms alone transfers the agent’s continuation and future execution authority. Active-request migration systems instead move a request that is currently executing [8]. During AgentShift’s target interval, no such request exists.

The problem is therefore not merely how to copy KV tensors. A correct relocation must satisfy four requirements. The next turn must execute on a different engine; it must obtain a full prefix hit; transfer should finish before the tool makes the agent runnable; and at most one executor may advance each agent step. Copying only KV leaves ownership ambiguous. Moving only control state causes destination re-prefill. Starting the copy after tool return exposes migration on the next turn’s critical path.

AgentShift treats the completed-turn boundary as a se-

mantic handoff point. At this boundary, the prefix is immutable and the agent cannot execute, which makes background transfer both cheaper and easier to order than arbitrary live migration. AgentShift first records a durable continuation containing committed progress, the current owner and epoch, pending tool state, output position, and managed-effect status. It then pins the source prefix, reserves destination KV slots, copies tensor-parallel shards, and publishes the destination entry only after every rank and layer completes. Finally, it atomically changes the owner epoch and rebinds the tool mailbox. The old owner loses authority at commit; its KV shadow remains until the destination emits a first-token acknowledgement.

This design turns an otherwise idle tool interval into migration slack. At 32K tokens, stateless rerouting takes 1260.5 ms after tool completion, and the same direct migration started on return takes 126.1 ms. Starting the handoff during a 500 ms blocked interval reduces the exposed delay to 52.4 ms, matching Sticky at 54.4 ms. In an eight-agent simultaneous return, AgentShift moves half the owners and completes in 730.9 ms, compared with 5579.8 ms for rerouting and 1250.4 ms for on-return migration. These gains do not rely on calling locality-only policies slower: Sticky, TTL, and source-side offload can approach the same latency only by leaving future execution on the source.

AgentShift also exposes a useful elasticity primitive. A model-loaded target is not useful to a warm agent until its state and execution authority are ready. In our warm-pool experiments, AgentShift relocates half of eight agents during scale-out and drains all eight owners during scale-in without historical prefill. This capability complements model-loading systems such as BlitzScale and Serverless-LLM [3, 11]; it does not replace cold model provisioning.

This paper makes three contributions:

1. **Agent continuation and ownership.** AgentShift separates authoritative progress and managed effects from executor-local acceleration state. Epoch-based ownership and step claims permit one executor to advance an agent, including across tool-result races and recovery.
2. **Completed-prefix mobility.** AgentShift transfers and atomically installs an immutable completed prefix across identically configured SGLang engines. It preserves full prefix hits for Qwen3-8B and Qwen3-32B at tensor parallelism 1–4.
3. **Gap-aware semantic handoff.** AgentShift overlaps prefix transfer with tool-induced blocking, commits ownership only after destination readiness, and retains a source shadow until first-token acknowledgement. The result combines sticky-like next-turn latency with changed future placement.

2 Background and Motivation

2.1 Cross-Turn State Creates Placement Stickiness

An autoregressive LLM stores one key and one value vector per cached token, layer, and attention head. Multi-turn agents repeatedly reuse nearly their entire history, so a completed prefix can represent gigabytes of GPU memory and hundreds of milliseconds of avoided prefill. SGLang’s RadixAttention [12] shares such prefixes through a radix tree and naturally favors routing the next turn to the engine containing the longest match.

This optimization couples locality to placement. Let agent a finish a turn on source engine s with completed prefix P_a . While a waits for a tool, P_a is useful but no request consumes compute. If the next turn remains on s , it receives a full hit but joins s ’s queue. If it moves to destination d , a stateless router reconstructs P_a before decoding. The penalty increases with agent age because $|P_a|$ grows across turns.

Figure 1 characterizes this state using two public trace sources. The Kimi K2.5 request trace contains 245,555 input lengths but no session identifiers; it supports context-length claims, not agent lifetimes. The FlowPrefill trace contains parent-linked turns; it supports prefix growth and clustered child-arrival measurements. Its inter-turn deltas are return proxies, not labeled tool durations. We preserve these schema boundaries rather than infer unavailable semantics.

2.2 Existing Actions Leave a Capability Gap

Existing serving mechanisms expose three relevant actions. Routing selects an engine for the next request. Residency management retains, evicts, offloads, or prefetches KV. Request migration moves an active inference request. These actions are valuable, but none defines how a suspended agent transfers both its warm prefix and its right to continue.

Table 1 states the capability boundary. Sticky, Agentix-style routing, Continuum-style TTL, and TokenCake-style source reload can preserve locality only while execution remains on the source. Stateless rerouting changes placement but loses the prefix. Symphony-style prefetch can make KV available at a destination, but state availability alone does not fence a stale source. Lumnix migrates active requests; the completed-turn interval has no active request to migrate. On-return handoff satisfies state and ownership requirements, but begins only when the agent is already runnable.

The fair objective is consequently constrained. AgentShift minimizes post-tool latency subject to destination execution, a full prefix hit, and single-owner progress. Locality-only baselines provide a latency floor but do not satisfy destination execution. Stateless rerouting provides a load-balancing reference but not a locality-preserving alternative. On-return migration is the closest controlled baseline because it uses

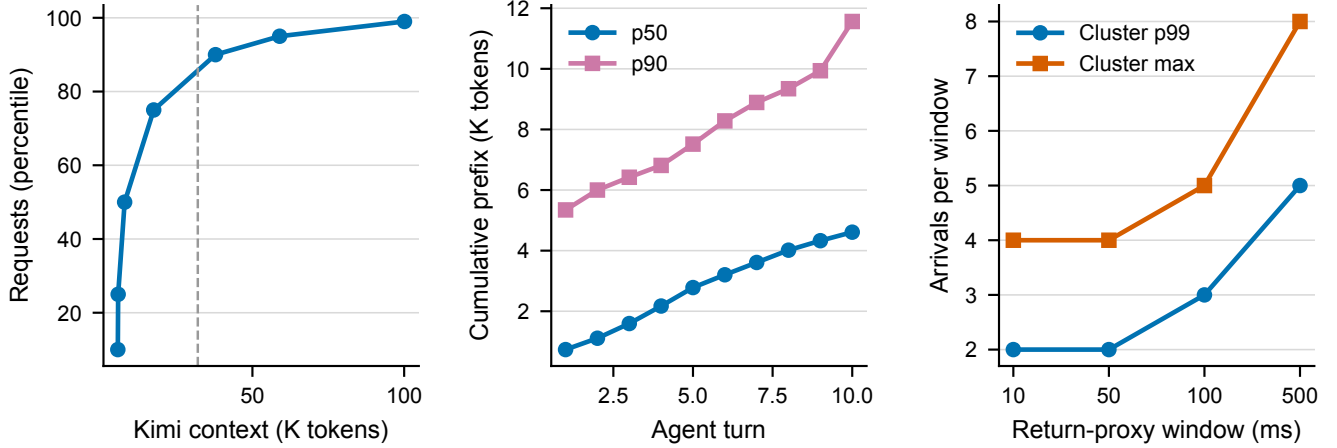


Figure 1: Workload evidence. Kimi contains substantial long-context demand; FlowPrefill sessions accumulate context across turns; and child-arrival proxies cluster in short windows. FlowPrefill deltas are not labeled tool durations.

Table 1: Capability comparison. Literature-named baselines in our experiments are mechanism-equivalent implementations, not official artifact reproductions.

Design	Moves next turn	Full prefix hit	Prepares in gap	Durable owner transfer
Sticky	No	Yes	N/A	No
Agentix-style routing	Optional	Only on source	No	No
Continuum-style TTL	No	Until expiry	No	No
TokenCake-style source reload	No	Yes	Yes	No
Stateless reroute	Yes	No	N/A	Routing only
Symphony-style shared prefetch	Yes	Yes	Yes	No
Llumnix active migration	Yes	Yes	N/A	Request scoped
On-return handoff	Yes	Yes	No	Yes
AgentShift	Yes	Yes	Yes	Yes

the same engines, tensors, transport, installation, and ownership protocol; only its start time differs.

2.3 Design Principles

AgentShift follows two principles. First, durable progress is authoritative. A database record determines the committed step, owner, epoch, pending tool, stream position, and managed effects. Executor memory cannot overrule it. Second, KV is reconstructable acceleration state. A prefix may be copied, shadowed, or discarded without changing agent semantics, provided it is never made visible before complete installation and recovery preserves one valid owner.

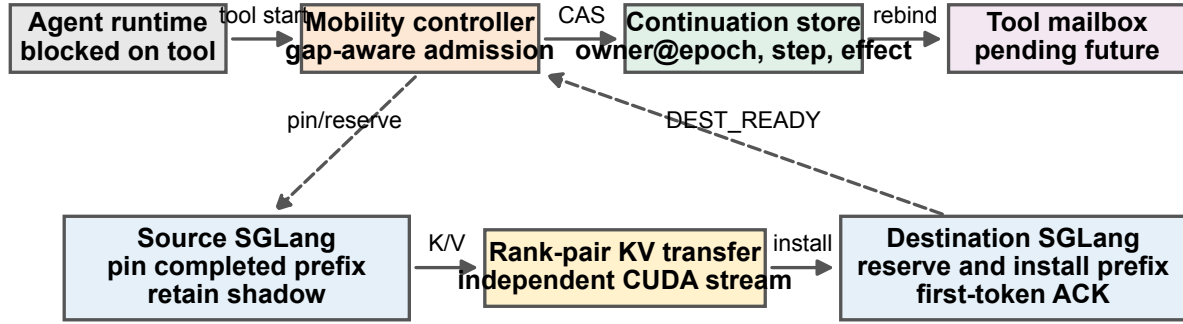
These principles separate correctness from performance. Ownership commit decides who may execute. Prefix installation decides whether that execution is fast. A source shadow improves recovery latency but is not the source of truth. This separation lets AgentShift abort safely before commit, recover deterministically after commit, and fall back to reconstruction when no warm copy survives.

3 AgentShift Overview

AgentShift operates between two completed LLM turns. The source and destination run the same model revision, tokenizer, KV layout, and tensor-parallel degree. The current prototype targets multi-head attention and identical tensor-parallel rank mappings. These compatibility constraints allow direct shard-preserving copies; heterogeneous layouts would require KV transformation.

Figure 2 shows the handoff. When an agent blocks on a tool, the runtime records the pending future in the continuation store. The mobility controller selects a destination, pins the source prefix, reserves destination slots, and starts an asynchronous rank-to-rank copy. Each destination rank validates the complete token key and reports readiness only after all layers are installed. The controller then performs one compare-and-swap from (s, e) to $(d, e + 1)$ and rebinds the mailbox. A tool result arriving before or after this operation is consumed by the owner of the current epoch.

The protocol has two safety invariants. **Single advancement:** for every $(agent, step)$, at most one current-epoch ex-



Completed-turn boundary: immutable prefix, no active decode request

Figure 2: AgentShift separates durable continuation from warm KV state. It copies an immutable completed prefix during the tool gap, publishes it only after all ranks are ready, commits ownership with an epoch CAS, and releases the source shadow after first-token acknowledgement.

ecutor may claim and advance the step. **Atomic visibility:** a destination prefix is not visible to lookup until every required tensor-parallel shard and layer has been installed under the exact token key. Together, these properties prevent a partial cache entry from being consumed and prevent two valid executors from using two complete copies.

The ownership CAS is the commit boundary. Before commit, the source remains authoritative and a failed preparation is discarded. After commit, the destination is authoritative even if the source still holds a shadow. If the destination fails before its first token, the controller increments the epoch and recovers from the source shadow. Once first-token acknowledgement releases the shadow, failure recovery remains correct but may reconstruct the prefix.

4 Agent Continuation and Ownership

Moving an agent requires a durable description of what may happen next. AgentShift represents this description as an `AgentContinuation`: agent identifier, committed step, owner engine, monotonic owner epoch, complete token IDs, pending future identifier, workspace reference, output stream offset, and application metadata. This record is intentionally smaller than a process checkpoint. AgentShift does not move a Python stack, shell, browser, or tool process; it records the logical state needed to issue the next model turn after the external result becomes available.

4.1 Epoch-Fenced Execution

Every inference turn carries an owner pair $(engine, epoch)$. Before sending work to SGLang, the managed executor compares this pair with the durable continuation and claims the next $(agent, step)$ under a unique request identifier. A

uniqueness constraint admits one live claim. Completion atomically records the new token continuation and marks the claim complete. Requests from an old engine or epoch fail before inference, and retries with a conflicting request identifier fail before a second step can be advanced.

Ownership changes only through a compare-and-swap. For a migration from source s at epoch e to destination d , the store accepts the update only if the migration is `DEST_READY` and the continuation still names (s, e) . A successful transaction sets the owner to $(d, e + 1)$ and the migration to `COMMITTED`. This ordering gives each engine a simple rule: an executor may act only while its pair equals the durable owner pair.

The protocol uses epochs rather than a boolean owner flag because recovery is also an ownership change. If d fails after commit, recovery advances to epoch $e + 2$ on the selected recovery engine. Delayed work from both $s@e$ and $d@(e + 1)$ is then stale. The protocol does not depend on synchronized clocks; ordering comes from store transactions and monotonic epochs.

4.2 Tool Results and Managed Effects

A global mailbox decouples tool completion from engine placement. Tool results are keyed by $(agent, step, future)$ and inserted idempotently. The current-epoch owner reads the result after claiming the next step. A result that races the ownership CAS therefore remains available without requiring the tool process to know the destination.

External side effects require a narrower contract. Operations routed through AgentShift receive a stable operation identifier and move through `PREPARED`, `SUBMITTED`, `COMPLETED`, or `UNKNOWN`. The proxy records `SUBMITTED` before invoking the external operation. It returns a stored result

for a completed retry, and it refuses to replay a submitted or unknown operation automatically. This provides at-most-once submission for managed effects; it does not provide exactly-once semantics for arbitrary third-party tools.

4.3 Abort, Recovery, and Restart

The migration state machine separates preparation from authority. `PREPARING`, `COPYING`, and `DEST_READY` are pre-commit states, so the source retains ownership. A failed copy marks the migration `ABORTED`, returns uninstalled destination reservations, and leaves the source prefix pinned only until cleanup completes. A controller restart can inspect durable state, wait for a still-active copy, or abort an uncommitted destination without changing the owner.

`COMMITTED` is the only post-commit state before source release. The destination owns future progress, while the old prefix remains a non-authoritative shadow. Destination first-token acknowledgement releases this shadow and changes the migration to `SOURCE_RELEASED`. A lost acknowledgement is harmless because release is idempotent and restart reconciliation observes the durable owner. Multiple acknowledgements to one source are serialized because SGLang’s tokenizer control communicator permits one in-flight RPC per operation.

Recovery preserves correctness even without a warm copy. Before source release, the controller can rebind the shadow to epoch $e + 2$ and resume with a full hit. After release, the same durable ownership transition chooses a recovery engine, but the engine reconstructs the prefix from token IDs. The shadow therefore reduces recovery cost; the continuation and epoch protocol provide correctness.

5 Completed-Prefix Mobility

Completed-prefix mobility turns engine-local KV into movable acceleration state. For a prefix of n tokens, L transformer layers, h_{kv} KV heads, head dimension d , and element width b , the unsharded MHA payload is $2nLh_{kv}db$ bytes. AgentShift preserves the existing tensor-parallel sharding, so each source rank sends its local rows directly to the corresponding destination rank.

5.1 Source Pinning and Destination Reservation

The source first validates the full token key against a resident RadixCache node and increments the node’s lock reference. Pinning prevents normal LRU eviction during transfer. The current implementation uses page-aligned completed turns; the continuation records the exact token sequence so the destination cannot install a semantically different prefix under the agent identifier.

The destination reserves request-to-token and token-to-KV slots before receiving data. It may evict only unlocked cache entries. Reserved slots remain visible to SGLang’s pool invariant checker even though they are not yet visible to prefix lookup. If capacity is unavailable, preparation fails before ownership changes.

5.2 Rank-to-Rank Copy and Atomic Installation

Corresponding source and destination TP ranks join persistent two-rank NCCL groups. A bounded worker copies one layer at a time on a separate CUDA stream, allowing scheduler RPCs to return while the data plane proceeds. The controller polls both engines; each engine aggregates status across its TP CPU group and reports the least advanced rank. Any rank failure fails the operation.

The destination installs the RadixCache entry only after all K/V rows have arrived. Installation inserts the exact complete token key and transfers allocation ownership to the normal cache lifecycle. This implements the visibility invariant:

$$visible(P_a, d) \Rightarrow \bigwedge_{r \in TP} \bigwedge_{l \in layers} installed(P_a, d, r, l).$$

A successful transfer retains installed rows when temporary transfer records are cleaned. A failed transfer frees reservations that never became visible. This distinction avoids both leaking failed allocations and reclaiming a valid cache entry after success.

5.3 Why Completed Turns

The completed-turn boundary removes write races from the data path. Unlike live request migration, the prefix does not grow during transfer, so AgentShift needs neither iterative dirty-state copying nor a decode pause. Unlike a shared KV store, the destination receives its normal local RadixCache representation and can immediately use the serving engine’s existing prefix lookup path.

This scope is deliberate. AgentShift does not migrate active decode, speculative draft KV, MLA state, Mamba state, or heterogeneous TP layouts. These cases require different consistency or transformation mechanisms. The current abstraction targets the interval common to tool-using agents: immutable model state, pending external work, and no active request.

6 Gap-Aware Semantic Handoff

Completed-prefix mobility removes re-prefill but does not by itself remove migration latency. If a copy starts only after the tool returns, the entire transfer delays the next turn.

AgentShift starts preparation while the agent cannot execute. For migration time T_m and remaining blocked interval T_g , the copy contribution exposed after tool return is

$$T_{\text{exposed}} = \max(0, T_m - T_g).$$

6.1 Eligibility and Placement

Only blocked agents with a completed resident prefix and no unresolved managed effect are eligible. The destination must use a compatible engine configuration and have enough free KV slots. AgentShift estimates copy time as a fixed setup cost plus KV bytes divided by measured bandwidth. It rejects candidates whose estimated exposure exceeds a configured budget.

The prototype uses an explainable cost-benefit score rather than a learned predictor. For agent a and destination d , positive terms estimate source queue relief, avoided re-prefill, and pressure-weighted HBM relief. Negative terms estimate interference and exposed copy time:

$$\text{Score}(a, d) = Q_{\text{relief}} + \lambda R_{\text{saved}} + \mu H_{\text{relief}} - \nu I_{\text{copy}} - \rho T_{\text{exposed}}.$$

The controller ranks positive candidates, reserves destination capacity, selects at most one destination per agent, and enforces a migration-concurrency limit. When admitted agents share a serial transfer channel, it considers them by return deadline; if cumulative handoff cost exceeds a deadline, it defers the longest handoff in the current set. This admissible-first order maximizes the predicted number prepared before return, rather than minimizing the tail of every agent. Exact tool completion prediction is not a correctness requirement. Prediction affects whether a copy finishes in the gap and whether its cost is worthwhile.

6.2 Handoff Ordering

The data plane and authority plane meet at `DEST_READY`. AgentShift starts destination receive before source send, waits for all-rank completion, cleans transfer bookkeeping, and persists `DEST_READY`. Only then does it commit the ownership CAS. This sequence prevents an owner from reaching a destination whose prefix is incomplete.

Tool completion does not interrupt a copy or bypass the owner check. If the tool returns early, its result waits in the mailbox and the next turn incurs the remaining transfer delay. If preparation fails, the source remains owner and can consume the result immediately. If commit wins the race, the destination consumes it under the new epoch.

The current NCCL path serializes migrations within one coordinator because point-to-point messages are untagged and source and destination queue order must match. The policy may admit multiple migrations, but the evaluated data path drains them through this bounded channel. Multiple

tagged groups or an RDMA backend are future transport improvements, not changes to the semantic protocol.

6.3 Warm-Pool Elasticity

Semantic handoff makes model-ready capacity useful to existing warm agents. During warm scale-out, the controller can move blocked owners to an already loaded engine before their next turns arrive. During scale-in, an engine enters `DRAINING`; it receives no new owners and hands off resident agents at completed-turn boundaries until its owner count reaches zero.

This primitive decomposes useful capacity into three events. A target is **model-ready** when its weights can execute requests, **state-ready** when the selected prefix is locally installed, and **authority-ready** when the ownership CAS has committed. Cold-start systems accelerate the first event [3, 11]. AgentShift connects the latter two for long-lived agents. Our evaluation therefore measures warm-pool elasticity and does not claim cold autoscaling.

7 Implementation

We implement AgentShift as a Python control plane and a modified SGLang data plane. The control plane stores continuations, migrations, mailboxes, step claims, and managed effects in SQLite with write-ahead logging and full synchronous commits. It exposes a managed executor, an asynchronous migration coordinator, placement policies, and mechanism-equivalent baseline coordinators. SQLite gives the prototype a concrete durable commit point; the protocol does not depend on SQLite-specific ordering beyond transactional compare-and-swap.

The data-plane changes extend SGLang’s RadixCache and KV allocator with pin, reserve, transfer, install, release, and epoch-rebind operations. HTTP control endpoints enqueue operations and return status. Persistent NCCL groups connect corresponding tensor-parallel ranks, and an independent CUDA stream performs layer-wise BF16 K/V copies. Rank-local results are aggregated before the controller observes completion.

The implementation is based on SGLang commit 034dd39189ba1ace1308d3c8a58df275ef301a21. We test Qwen3-8B and Qwen3-32B with TP=1, 2, and 4. The supported path uses MHA, RadixCache, page-aligned completed prefixes, identical model revisions, and identical TP layouts on one eight-H100 node. CUDA Graph is disabled consistently across compared strategies.

Implementation testing exposed two lifecycle races. First, concurrent first-token acknowledgements issued overlapping release RPCs to one engine; per-source locks now serialize those releases. Second, cache flush could reset RadixCache while AgentShift retained pinned-prefix metadata; flush now rejects active transfers and clears the registry before reset.

These fixes illustrate why prefix mobility must participate in the serving engine’s allocation lifecycle rather than copy tensor addresses out of band.

8 Evaluation

Our evaluation asks seven questions:

1. Does AgentShift preserve locality after changing placement?
2. How much migration can a blocked interval hide?
3. Does semantic handoff relieve a return hotspot?
4. Can it provide useful warm-pool elasticity?
5. How much does transfer interfere with foreground inference?
6. Does the protocol recover without double advancement?
7. How does the prototype generalize and where does its control plane saturate?

8.1 Experimental Setup

We run two SGLang engines on one server with eight NVIDIA H100 80 GB GPUs connected by NVLink. The main matrix uses Qwen3-8B in BF16 with TP=1 on GPUs 0 and 1. Each engine allocates 270K KV tokens and uses FA3 attention, FlashInfer sampling, page size one, and 4096-token chunked prefill. We also test Qwen3-8B at TP=2 and Qwen3-32B at TP=4. The software stack uses PyTorch 2.10, CUDA 12.8, NCCL 2.27.5, and our SGLang fork at commit 034dd3918.

All strategies use the same weights, prompts, output lengths, engines, and KV layout. Scenario order is randomized where supported, and transfer groups and tier allocators are warmed outside timed regions. Five repetitions support headline comparisons; wider matrices use three. CUDA Graph is disabled for every strategy, so relative comparisons are aligned but absolute latency requires production-graph revalidation.

We compare four primary mechanisms. **Sticky** runs the next turn on the source and defines the locality floor. **Reroute** sends the next turn to the destination without KV and reconstructs history. **On-return** uses AgentShift’s direct transfer, installation, and ownership protocol but starts after the tool returns. **AgentShift** starts the same handoff at tool submission. **Oracle** is a timing upper bound that knows the gap.

We also implement literature-inspired design points in the same SGLang testbed. The **program-aware router** trades measured cache savings against queue relief. **TTL** is calibrated from coding-tool durations and retains state on the source until expiry. **TokenCake-Source** offloads KV to host memory and reloads it to the source during the gap. **Shared**

Table 2: Next-turn latency for Qwen3-8B, TP=1, 32K prefix, 500 ms gap, and five repetitions.

Strategy	Mean (ms) ↓	Full hit	Owner moved
Sticky	54.4	Yes	No
Reroute	1260.5	No	Routing only
On-return	126.1	Yes	Yes
AgentShift	52.4	Yes	Yes
Oracle	54.5	Yes	Yes

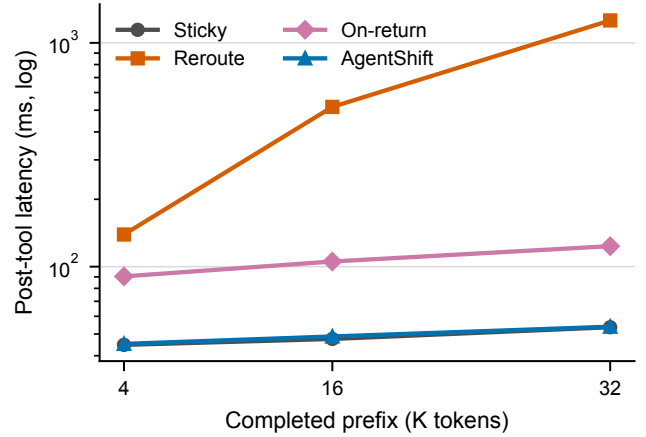


Figure 3: Post-tool latency as context grows. Reroute reconstructs a growing history, while AgentShift changes placement and remains near Sticky.

prefetch stages KV through local shared memory and imports it at the destination, with and without AgentShift’s ownership CAS. These are mechanism-equivalent implementations, not official Agentix, Continuum, TokenCake, or Symphony artifacts.

8.2 RQ1: Locality After Placement Change

AgentShift preserves the sticky latency floor after relocation. At 32K tokens and a 500 ms gap, its mean post-tool latency is 52.4 ms, compared with 54.4 ms for Sticky, 126.1 ms for On-return, and 1260.5 ms for Reroute (Table 2). The small difference from Sticky is run noise; both execute the next turn with a full local hit. AgentShift changes owner and placement, whereas Sticky does neither.

Relocation value grows with accumulated context. Figure 3 sweeps 4K, 16K, and 32K prefixes. AgentShift is 3.08x, 10.61x, and 23.36x faster than Reroute, respectively. The same sweep gives 2.00x, 2.16x, and 2.29x over On-return. Reroute grows with historical prefill; AgentShift remains near the cost of the incremental next turn.

The full prefix moves, not an approximate or partial cache entry. At TP=2, Qwen3-8B AgentShift takes 83.2 ms at 32K, versus 75.4 ms for Sticky and 704.3 ms for Reroute, an 8.46x gain over rerouting. At TP=4, Qwen3-32B takes 120.5 ms,

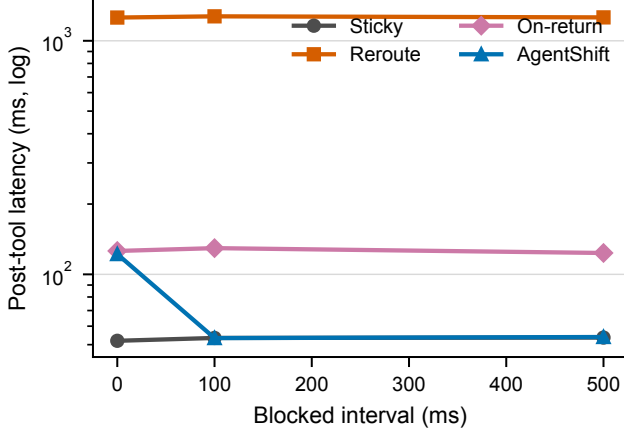


Figure 4: Post-tool latency versus blocked interval at 32K. AgentShift removes transfer from the critical path once the interval covers migration; On-return cannot use the interval.

versus 118.7 ms and 1637.3 ms, a 13.58x gain. The destination reports all 32,772 completed-prefix tokens as cached in the 32B run.

8.3 RQ2: Hiding Transfer in Blocked Gaps

Gap-aware timing provides a benefit distinct from KV mobility. On-return and AgentShift share the same 4.50 GiB 32K transfer, destination installation, and ownership CAS. At a zero-length gap their delays converge. At 100 ms, AgentShift reaches 53.4 ms, compared with 129.5 ms for On-return and 1272.7 ms for Reroute. At 500 ms it reaches 53.9 ms and matches Sticky at 53.6 ms. Figure 4 follows the expected $\max(0, T_m - T_g)$ relationship.

Three real commands confirm both the opportunity and its limit. A `git status --short` invocation lasts 8.9 ms and exposes most of the copy: AgentShift takes 113.2 ms after return, versus 131.5 ms for On-return. State-store and control-plane test commands last 403.4 and 488.2 ms; both fully hide transfer and reduce post-tool latency from 125.4 and 124.8 ms to 54.5 and 54.7 ms. AgentShift therefore needs admission, not an assumption that every tool is long.

The benefit persists across multiple tool turns. We replay eight coding agents for three turns, invoke real subprocess tools, and grow each prefix from 16K to 32K. AgentShift relocates four owners, preserves a full hit for every turn, and completes in 5.002 s. Sticky takes 7.169 s without moving an owner, Reroute takes 6.264 s and reconstructs 65,548 historical tokens, and On-return takes 5.318 s. AgentShift therefore improves makespan by 1.43x, 1.25x, and 1.06x, respectively. Its post-tool p95 is 1.711 s versus 1.708 s for On-return, so proactive overlap does not improve every turn in this controlled replay.

Ordering matters when blocked agents share one transfer stream. We combine six 4K–32K prefixes with decorrelated

60–500 ms gaps and calibrate end-to-end handoff cost using three samples per size. AgentShift’s admissible-first order completes 83.3% of handoffs before tool return, compared with 50.0% for FIFO and shortest-KV and 33.3% for earliest-return; all policies retain full hits. AgentShift defers one predicted-late migration, which improves coverage but increases that agent’s tail. We therefore optimize and report in-gap coverage separately from exposed latency.

Trace analysis indicates that long warm prefixes and clustered returns occur, but it does not establish a universal tool-gap distribution. In Kimi K2.5, 29.3% of 245,555 requests have at least 16K input tokens and 12.5% have at least 32K. Across 100K worker records, the median KV hit fraction is 63.3%. In FlowPrefill, the 90th-percentile prefix increases from 5.5K at turn one to 11.8K at turn ten; 500 ms child-arrival windows contain five returns at p99 and eight at maximum. FlowPrefill deltas are proxies because the trace does not label tool execution.

8.4 RQ3: Correlated-Return Hotspots

AgentShift removes the locality–balance choice in the tested return burst. We warm eight 32K agents on the source, block them for 500 ms, and return them simultaneously. The destination receives half of the next turns. AgentShift completes the burst in 730.9 ms with a full hit for all agents and transfers ownership for the relocated half. Reroute completes in 5579.8 ms and re-prefills 131,088 historical tokens; On-return takes 1250.4 ms.

Source-side offload does not solve the same hotspot. TokenCake-Source takes 2196.4 ms in this burst and leaves every next turn on the source. Sticky is fast at 783.7 ms but also moves no owner. AgentShift’s 730.9 ms should therefore be read as matching the locality floor while redistributing work, not as a general claim that it makes local execution faster.

TTL shows the same capability boundary under capacity pressure. We calibrate a 491.9 ms TTL from coding-tool measurements, with 88.9% coverage over nine held-out samples. Eight 16K agents compete with twelve 16K pressure requests. At a 400 ms gap, TTL retains all prefixes but leaves all owners on the source; AgentShift moves half and has similar makespan. At a 500 ms gap, TTL expires and preserves only 33% of prefixes. AgentShift retains all prefixes, improves makespan from 8048.6 to 5545.6 ms (1.45x), and improves mean post-tool latency from 6631.1 to 3129.7 ms (2.12x). Retain-or-evict and relocate are different scheduler actions.

Our local shared-memory path is not competitive with direct GPU transfer. At 32K/500 ms, mechanism-equivalent shared prefetch takes 4868.9 ms without ownership and 5241.0 ms with CAS. These numbers diagnose an extra host/shared-tier hop in this implementation; they do not estimate official Symphony. More importantly, adding CAS to

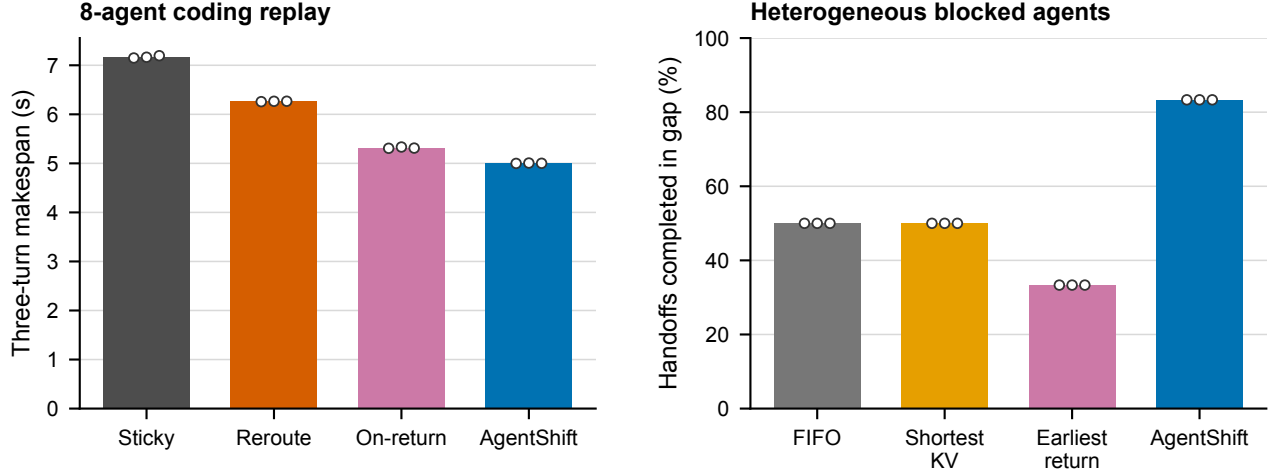


Figure 5: Controlled multi-turn and policy evaluation. Left: AgentShift moves half of eight coding-agent owners and has the lowest three-turn makespan. Right: admissible-first ordering raises in-gap completion coverage on a heterogeneous six-agent trace. Points show three repetitions.

shared prefetch shows that AgentShift’s semantic protocol can sit above another transport, while direct GPU mobility lowers the data path in our single-node setting.

8.5 RQ4: Warm-Pool Elasticity

AgentShift turns model-ready capacity into warm-agent capacity. In scale-out, we add an already loaded destination and select four of eight 16K agents during a 500 ms gap. AgentShift moves 50% of owners with 100% full hits and no re-prefill. Its post-tool makespan is 0.704 s, compared with 0.894 s for On-return and 2.616 s for semantic rerouting. Sticky takes 0.714 s but activates none of the new capacity.

Scale-in exposes a stronger semantic distinction. AgentShift hands off all eight owners and drains the source in 0.712 s without historical prefill. On-return takes 1.101 s, and semantic rerouting takes 4.583 s while reconstructing 131,104 tokens. Sticky takes 0.710 s but cannot drain: all eight owners remain. Thus AgentShift is 1.55x faster than On-return and 6.43x faster than rerouting among strategies that complete the drain.

These results establish stateful warm elasticity, not cold autoscaling. The target model is loaded before timing. A cold deployment would additionally pay weight-loading and engine-initialization cost; BlitzScale-like model provisioning could shorten that phase, after which AgentShift would prepare warm agent state and authority.

8.6 RQ5: Foreground Interference

Asynchronous transfer bounds, but does not eliminate, foreground stalls. We overlap one 32K migration with eight streaming decodes and submit a one-token arrival probe 5 ms

after transfer begins. Without migration, throughput is 397.7 token/s and arrival TTFT is 53.9 ms. Synchronous transfer lowers throughput to 397.0 token/s and increases TTFT to 78.0 ms. Asynchronous transfer sustains 397.9 token/s and reduces TTFT to 64.5 ms.

Steady-state token latency remains similar: TPOT p95 is 20.55 ms without migration, 20.42 ms with synchronous copy, and 20.57 ms with asynchronous copy. The maximum token gap is more sensitive, rising from 41.62 to 66.17 ms under synchronous transfer and to 53.38 ms under asynchronous transfer. Async copy therefore reduces excess arrival TTFT from 24.1 to 10.6 ms and excess maximum token gap from 24.6 to 11.8 ms. This experiment covers one migration; concurrent-copy admission remains a scaling limitation.

8.7 RQ6: Fault Semantics and Recovery

All eight cases in the real-engine fault campaign satisfy their expected invariants (Table 3). The campaign covers post-commit destination failure, failure after source release, lost acknowledgement, controller restart before CAS, a mailbox/CAS race, a managed SQLite effect, no-fencing ablation, and cache-flush lifecycle cleanup. The pre-commit case copies all 16,388 tokens before injecting controller failure at `DEST_READY`; recovery aborts to the source with a full hit. Faults combine real KV operations with deterministic control-plane injection; they are not independent-node crash-stop experiments.

Owner fencing is necessary for semantic handoff. In the router-only ablation, both source and destination accept the same next step. With epoch and step fencing, exactly one executor accepts it. The managed-effect case inserts one row

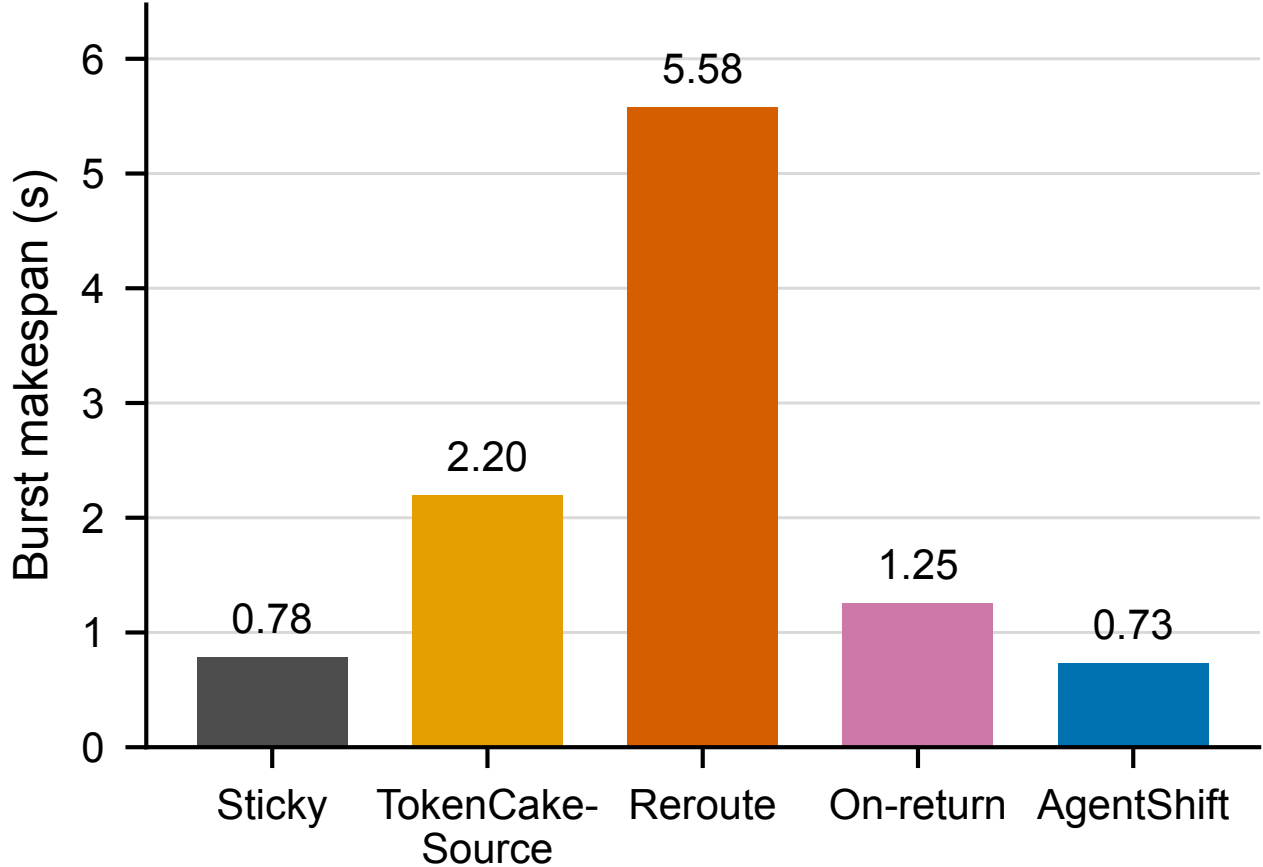


Figure 6: Eight-agent 32K return burst. AgentShift moves 50% of owners with full hits and reaches the low-latency, high-relief region. Source-only methods do not change future placement.

into an external SQLite table and records one submission call across retry.

The source shadow improves availability rather than correctness. At 16K, recovery before release reuses a full 16,388-token shadow and resumes in 48.1 ms. Recovery after release reconstructs the prefix and takes 523.2 ms. The shadow reduces this recovery point by 10.9x, while both paths advance ownership to a new epoch and reject stale executors.

8.8 RQ7: Control-Plane Scale

The prototype controller handles more ownership events than the current serialized data plane, but SQLite write tails limit production scale. With 100K registered agents, a single client performs 1105 ownership CAS operations/s with 1.8 ms p99. At 32 clients, throughput is 806 operations/s but p99 reaches 930 ms because full-synchronous writes serialize. Read paths remain much faster.

This result supports SQLite as an executable fault model, not as a cluster-scale metadata service. A production deployment would shard agent records or use a replicated trans-

actional store while preserving the same compare-and-swap and epoch semantics. The current controller is also a single failure domain; restart recovery is deterministic, but controller high availability is outside this prototype.

9 Discussion and Limitations

Single-node transport. AgentShift’s semantic protocol names independent engines, but our evaluation places them on one NVLink-connected host. We therefore make no claim about cross-node RDMA latency, network contention, or independent machine failures. A cross-node backend must preserve all-rank completion and destination visibility ordering. The useful follow-up experiment is not merely peak bandwidth; it must measure exposed handoff time, foreground network interference, and failure across node boundaries.

Compatible engines. Direct shard-preserving mobility requires the same model revision, tokenizer, attention layout, and TP degree. This matches homogeneous replicated serving pools but excludes heterogeneous TP resizing. Supporting a different TP degree requires KV resharding, and

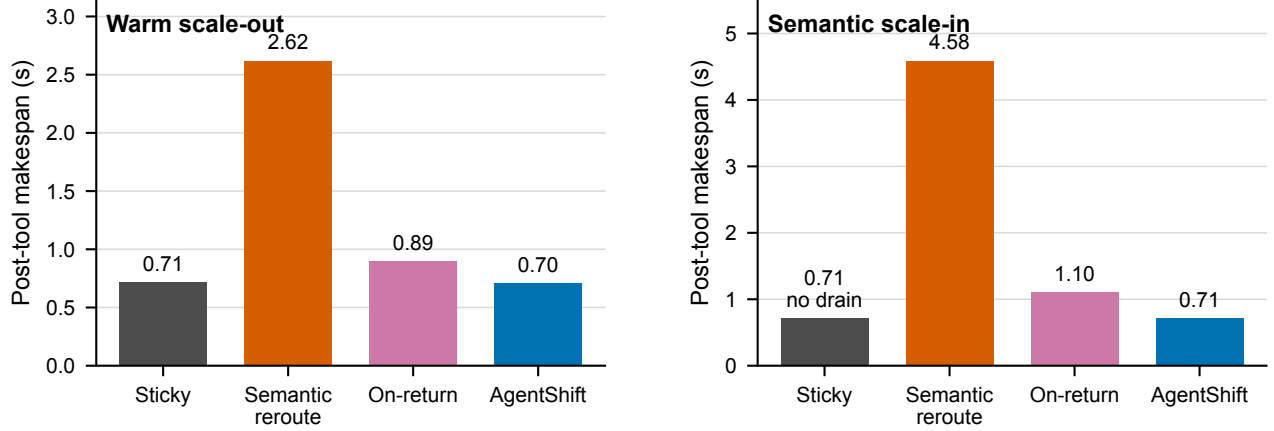


Figure 7: Warm-pool elasticity with eight 16K agents. AgentShift activates half of a model-ready target during scale-out and reaches zero source owners during scale-in without re-prefill. Sticky latency is low because it performs neither operation.

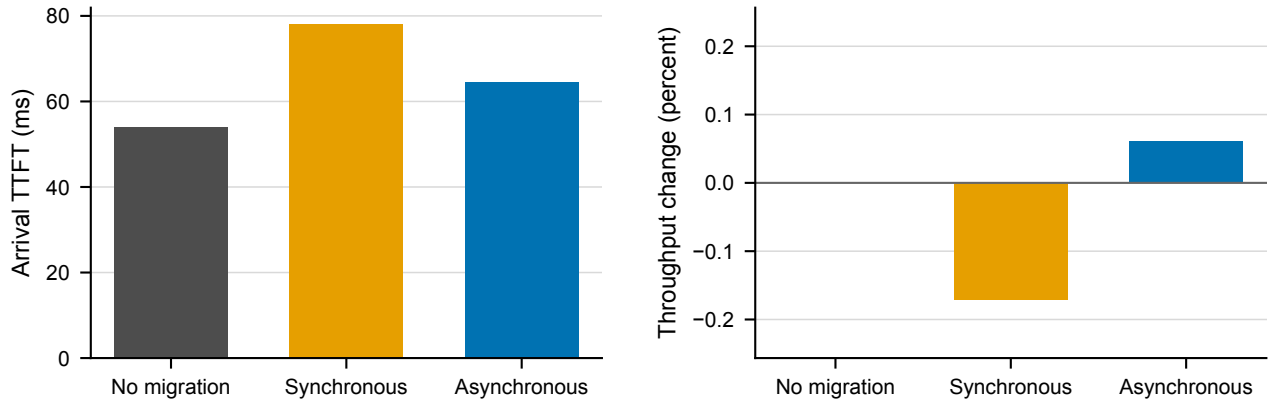


Figure 8: Foreground impact of one 32K migration. Async copy leaves throughput and steady TPOT within noise, while reducing excess arrival TTFT and maximum token gap relative to synchronous copy.

supporting MLA, recurrent state, or speculative decoding requires architecture-specific state definitions.

Completed turns. AgentShift targets an immutable prefix between LLM requests. It does not replace live migration for active decode. The two mechanisms can coexist: Llumnix-like migration handles a request already consuming compute, while AgentShift handles a continuation blocked on an external event. A scheduler could select the semantic boundary when available and use live migration only when immediate relief is necessary.

Warm rather than cold elasticity. AgentShift cannot execute on an engine until its model is loaded. Our elasticity experiment begins with model-ready capacity and measures state and authority readiness. Systems such as BlitzScale or ServerlessLLM can create capacity; AgentShift can then move long-lived warm agents onto it. Joint scheduling of model weights and agent KV is promising but may introduce network competition absent from the current results.

Prediction and concurrency. A short tool can expose

transfer and make proactive movement unattractive. The controlled coding replay and six-agent policy trace test repeated turns and heterogeneous gaps, but they are not full SWE-agent or function-calling benchmarks. The NCCL data path also serializes untagged transfers. Prediction-error sweeps, complete framework replays, and concurrent transfer groups are necessary to quantify cluster-level opportunity and wasted work.

Fault model and effects. The fault campaign validates durable ordering, retries, and recovery within one host using real engine operations and deterministic injection. It does not emulate every crash-stop boundary on separate machines. Managed effects receive at-most-once submission when invoked through AgentShift; arbitrary tools remain responsible for their own idempotency or transactional semantics.

Table 3: Fault-injection matrix. Claims apply within this evaluated fault model.

Fault point	Expected result	Observed
Destination fails after commit	Recover source shadow at new epoch	Pass
Destination fails after release	Cold reconstruct at new epoch	Pass
First-token ACK lost	Retry idempotent release	Pass
Restart at <code>DEST_READY</code>	Abort; source remains owner	Pass
Tool result races CAS	New owner consumes once	Pass
Managed effect retry	One external row/submission	Pass
No-fencing ablation	Full system admits one executor	Pass
Flush after terminal transfer	No stale prefix reference	Pass

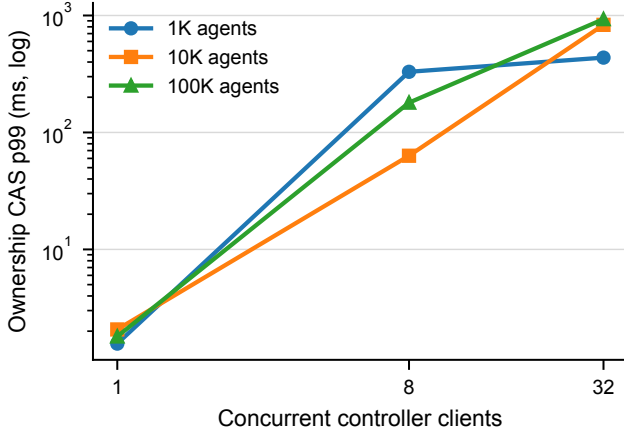


Figure 9: Control-plane behavior as registered agents and client concurrency increase. Aggregate throughput remains adequate for the prototype, but concurrent durable writes create a long p99 tail.

10 Related Work

10.1 Agent Serving and Program-Aware Scheduling

Agent schedulers use program structure to improve fairness, queueing, and locality. Agentix treats an agent program as the scheduling object and uses accumulated program service and locality-aware routing to place future calls [6]. This context improves the decision, but routing still chooses between a source with KV and a destination without it. AgentShift changes the available action by relocating completed state and ownership before the next call exists.

Request schedulers address a complementary scope. FastServe uses token-level preemption and proactive KV offload to reduce head-of-line blocking among active generation requests [10]. vLLM [4] and SGLang [12] improve memory utilization and prefix reuse. These systems optimize active request execution or local cache management; AgentShift adds a cross-turn object whose request has ended but whose future state remains live.

10.2 KV Retention, Offload, and Prefetch

Continuum assigns cache TTLs from tool duration and re-computation costs, retaining useful KV while controlling residency pressure [5]. TokenCake uses function-call stalls to offload and predictively reload inactive KV [2]. Both exploit the same blocked interval as AgentShift, but their primary action changes memory residency rather than future engine ownership. AgentShift can move execution while preserving a local hit at the destination.

Disaggregated KV systems make state accessible beyond one compute engine. Symphony uses advisory requests to prefetch session state from a memory tier and reduce stateful load imbalance [1]. LMCash and Mooncake expose related reusable KV or disaggregated transfer substrates [7, 9]. These systems can provide a transport or storage backend for AgentShift. Their state-availability mechanisms do not by themselves define step claims, owner epochs, mailbox re-binding, or managed-effect fencing.

10.3 Migration and Elastic Serving

Llumnix migrates active LLM requests and overlaps append-only KV transfer with continued decoding [8]. Serverless inference systems instead reduce model readiness time. ServerlessLLM caches weights across storage tiers [3], while BlitzScale multicasts parameters and uses cooperative execution during live scale-out [11]. AgentShift targets the gap between these abstractions: the model can already run, no inference request is active, but a long-lived agent’s accumulated state and authority still bind it to one engine.

The distinction is operational rather than competitive. Active migration, model provisioning, shared KV storage, and AgentShift can compose. AgentShift contributes the semantic boundary and commit protocol needed to turn available state and model capacity into one valid continuation owner.

11 Conclusion

AgentShift makes suspended-but-warm agent execution mobile across LLM engines. Its key insight is that a completed turn followed by an external wait provides an immutable

prefix and an interval in which the agent cannot execute. AgentShift uses this boundary to transfer the completed prefix, publish it atomically, and commit an epoch-fenced ownership handoff before the next turn becomes runnable.

The prototype preserves full prefix hits across Qwen3-8B/32B and TP=1–4. At 32K tokens, it matches Sticky while reducing post-tool latency by 24.04x over rerouting and 2.41x over on-return migration. It also relocates owners under a correlated return burst, provides warm-pool scale-out and drain, and recovers without double advancement within the evaluated fault model.

The current system is limited to compatible engines and completed turns on one node. Cross-node transport, labeled end-to-end agent replays, concurrent migration scheduling, and a replicated control store are the next steps. The broader result is independent of those extensions: warm KV availability is insufficient for stateful agents unless the serving system also knows which continuation the state accelerates and which executor may use it.

References

- [1] Saurabh Agarwal, Bodun Hu, Anyong Mao, Aditya Akella, and Shivaram Venkataraman. SYMPHONY: Enabling Compute-Memory disaggregation in LLM serving systems. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 2027–2041, 2026.
- [2] Zhuohang Bian, Feiyang Wu, Zhuoran Li, Teng Ma, and Youwei Zhuo. TokenCake: A KV-Cache-Centric serving framework for LLM-Based multi-agent applications. *arXiv preprint arXiv:2510.18586*, 2025.
- [3] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: Low-Latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153, 2024.
- [4] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 23)*, pages 611–626, 2023.
- [5] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph E. Gonzalez, and Ion Stoica. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.
- [6] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Agentix: An efficient serving engine for LLM agents as general programs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 2443–2459, 2026.
- [7] Kexin Qin et al. Mooncake: A KVCache-Centric disaggregated architecture for LLM serving. *arXiv preprint arXiv:2407.00079*, 2024.
- [8] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 173–191, 2024.
- [9] LMCache Team. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *Technical report*, 2024.
- [10] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. FastServe: Iteration-Level preemptive scheduling for large language model inference. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 57–74, 2026.
- [11] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. BlitzScale: Fast and live large model autoscaling with O(1) host caching. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 275–293, 2025.
- [12] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.